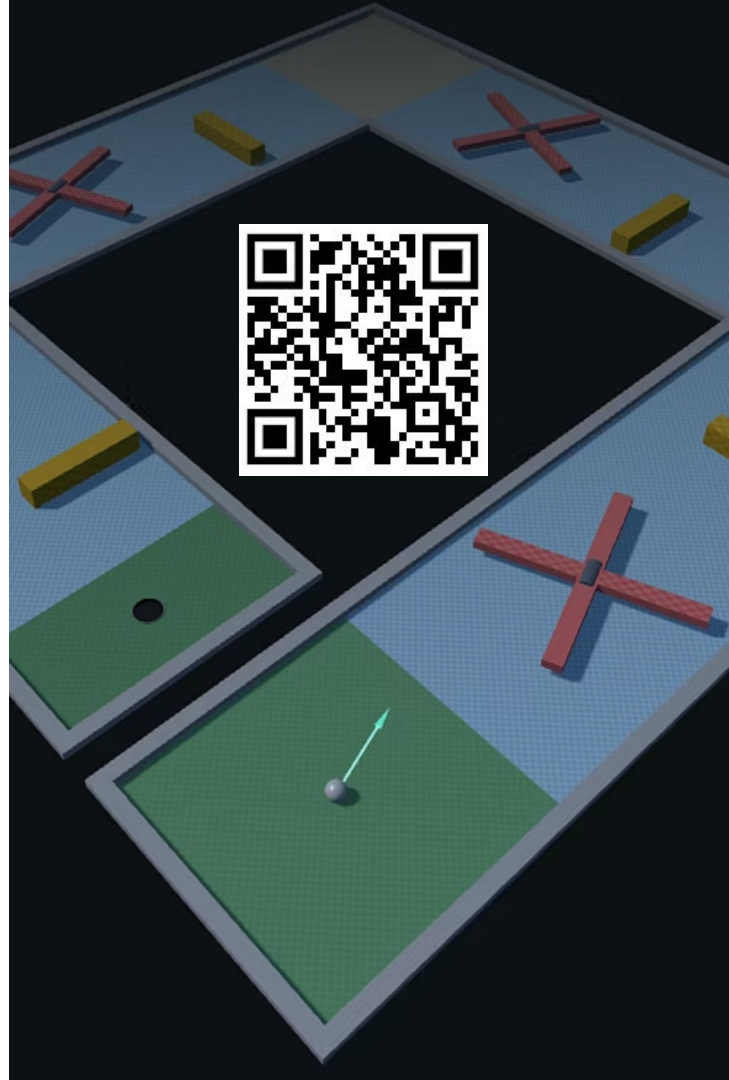
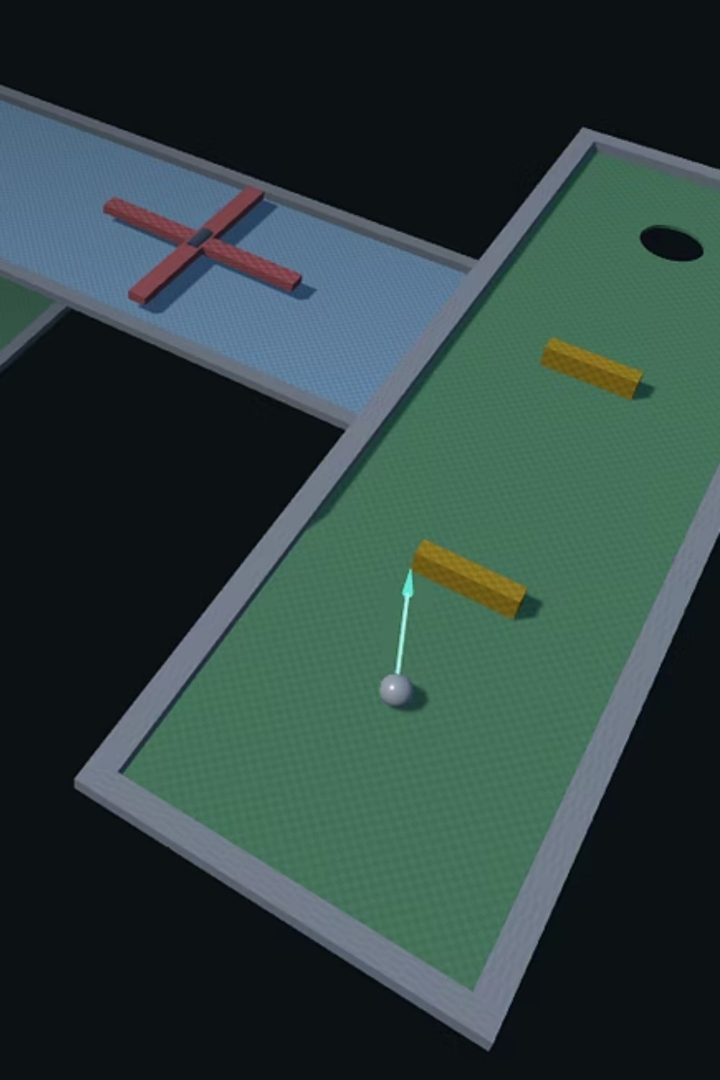


Mini-Golfe 3D

Projeto Final de Introdução à Computação Gráfica · 2025/2026

Tomás Cruz · github.com/tomascruz39/ICG-Projeto-Final ·
<https://tomascruz39.github.io/ICG-Projeto-Final/>





O Projeto

Jogo de **mini-golfe 3D**: colocar a bola no buraco no menor número de tacadas. Cinco mapas de dificuldade crescente, cada um com mecânicas únicas e identidade visual própria.

5 Mapas

Temas distintos – com obstáculos e mecânicas únicos

Interação Completa

Escolha de mapa, ajuste de mira e força, registo de recordes e modo Free Cam

Three.js Core

Apenas o núcleo da biblioteca - r164

Física de Raiz

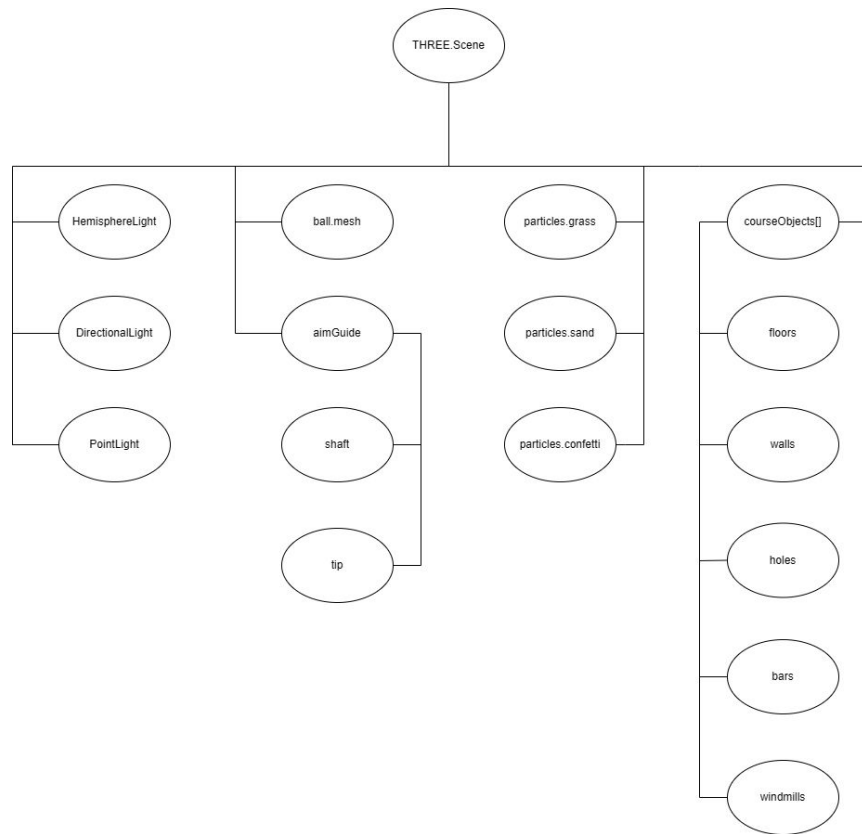
Motor de física e deteção de colisões implementados do zero, sem dependências externas

Modelos & Scene Graph

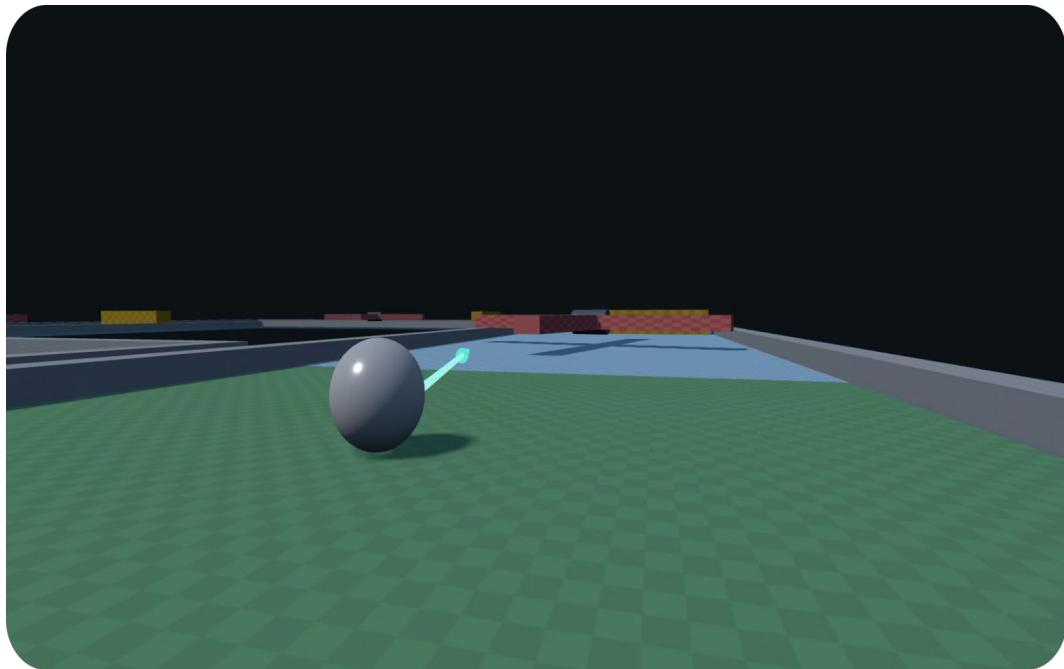
Todos os modelos são **primitivas geradas por código**. Luzes, bola, mira e partículas **persistem** entre mapas; apenas `courseObjects` é reconstruído a cada mudança de nível.

i Os dados de colisão estão **separados dos meshes** – arrays independentes geridos pelo motor de física.

- Bola → `SphereGeometry`
- Pisos com buraco → `ExtrudeGeometry`
- Buraco → `TorusGeometry` + `CylinderGeometry`
- Moinhos → `Group`: eixo + 2× pás
- Partículas → `Points` + `BufferGeometry`



Iluminação, Materiais & Texturas



Iluminação

- **HemisphereLight** — céu azulado + reflexo do chão, luz ambiente realista
- **DirectionalLight** — sol principal, única que lança sombras
- **PointLight** — preenchimento azulado

Materiais & Texturas

- **MeshStandardMaterial** — roughness/metalness, reage à luz e sombras
- **MeshBasicMaterial** — seta de mira, sempre visível e brilhante
- Texturas **100% procedurais** via `CanvasTexture`

Animação

Tudo procedural, dependente do tempo (não há keyframes nem `AnimationMixer`). Loop a ~60 fps via `requestAnimationFrame`; delta-time → independente do framerate, limitado a 33 ms

1

Barras deslizantes

movimento sinusoidal → $pos = base + \sin(t \cdot speed + fase) \cdot amplitude$

2

Moinhos rotativos

rotação linear → $rotation.y = t \cdot speed$

3

Bola

- integração de Euler: $pos += vel \cdot dt$
- gravidade: $vel.y -= 16 \cdot dt$
- atrito por decaimento exponencial: $vel := e^{(-drag \cdot dt)}$

4

Movimento da bola

`rotateOnWorldAxis` num eixo perpendicular à velocidade, $\text{ângulo} = (velocidade \cdot dt) / \text{raio}$

5

Partículas

cada partícula tem velocidade/idade/tempo de vida, atualizadas por frame

Interação do Utilizador

Câmara

PerspectiveCamera (FOV 60°). Modo orbital centrado na bola para inspecionar a tacada; modo **Free Cam** em 1.ª pessoa para analisar o level design.

Controlos Desktop

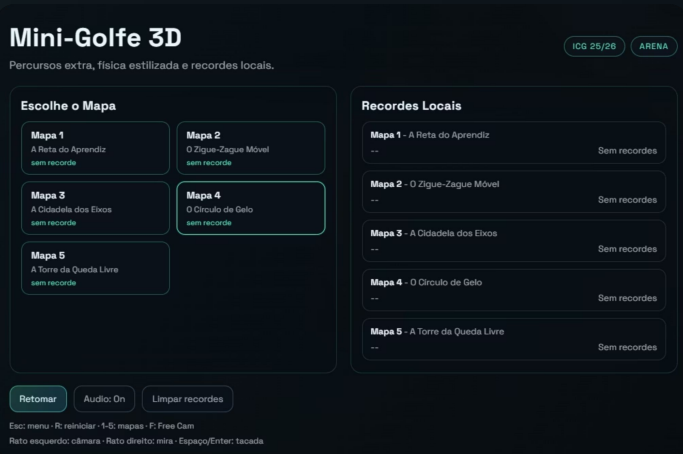
Rato esquerdo = rodar câmara · direito = ajustar mira · scroll = zoom. Teclado: W/S força, Espaço = tacada, R = reiniciar, F = Free Cam, 1-5 = mapas, M = Menu.

Mobile

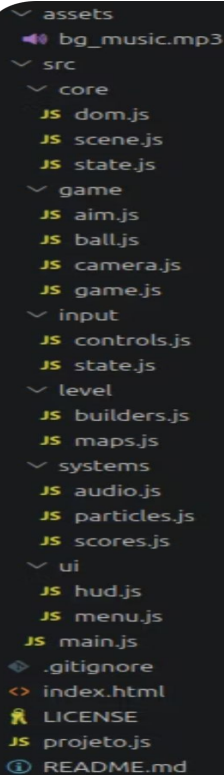
Detetado automaticamente. **PointerEvents** unificam rato/toque. UI tátil com **pinch-to-zoom**, slider de força e botão de alternar câmara/mira.

Menu

Escolha de mapa, visualização de recordes, controlo de áudio e opção de limpar progressos guardados.



Desenvolvimento



```

  assets
  bg_music.mp3
  src
    core
      dom.js
      scene.js
      state.js
    game
      aim.js
      ball.js
      camera.js
      game.js
    input
      controls.js
      state.js
    level
      builders.js
      maps.js
    systems
      audio.js
      particles.js
      scores.js
    ui
      hud.js
      menu.js
  main.js
  .gitignore
  index.html
  LICENSE
  projeto.js
  README.md

```

Ambiente & Organização

JS ES6 nativo. Three.js carregado via `importmap` do CDN. Ficheiros estáticos servidos via **GitHub Pages**. Editor: VS Code.

Estrutura modular: `core/` · `game/` · `input/` · `level/` · `systems/` · `ui/`

Destaque Técnico

Motor de física próprio: colisão esfera-AABB, atrito por superfície, gravidade e obstáculos móveis com transferência de momento.

Dificuldades

- Evitar *clipping* quando a bola é esmagada entre obstáculos
- Normais dos moinhos em rotação contínua
- Detecção fiável de "bola parada"

Uso de IA

Google Gemini

Estruturação da **matemática da física**. Suporte em *debugging* de comportamentos inesperados da bola. Integração mobile.

GitHub Copilot

Autocompletar funções repetitivas (builders dos mapas), sugestões de *refactor* para melhorar a legibilidade do código existente e otimizações do jogo.

Decisões Humanas

O **design dos mapas** e as **decisões de arquitetura** foram inteiramente do autor. A IA assistiu na execução, não na conceção.

Performance & Conclusões

Otimizações

- **Caches partilhadas** de geometria, material e textura → menos VRAM
- `pixelRatio` limitado a $2 \times$ · `PointLight` sem sombra
- Partículas com **typed arrays** e vetores temporários reutilizados
- `fog` para esconder geometria distante

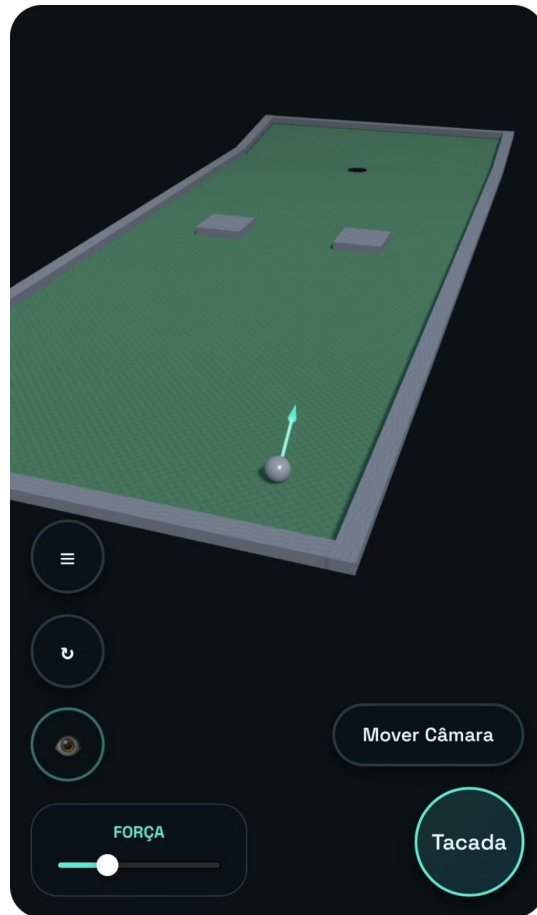
Conclusões

Compatível com desktop e mobile.

Destaques: motor de física de raiz, texturas procedurais e áudio sintetizado.



Futuro: migrar colisões hardcoded para dados genéricos e criar um sistema de níveis baseado em dados.



Referências

Three.js

Documentação oficial — threejs.org. Base de toda a renderização 3D, geometrias, materiais e sistema de cena.

Material das Aulas de ICG

Transformações geométricas, viewing, iluminação/shading, texturas e meshes — fundamentos teóricos aplicados no projeto.

MDN Web Docs

Web Audio API (áudio sintetizado) e Pointer Events (unificação de rato/toque para suporte mobile).

Space Grotesk — Google Fonts

Tipografia utilizada na interface do jogo e nos elementos de UI.

Música: *aquatic ambience* — scizzie

Faixa de ambiente utilizada no menu e durante o jogo.